```
BASE = "https://pokeapi.co/api/v2"


def buscar_pokemon(nome):
    url = f"{BASE}/pokemon/{nome.strip().lower()}"
    try:
        resposta = requests.get(url, timeout=10)
        resposta.raise_for_status()
        return resposta.json()
    except requests.exceptions.HTTPError:
        if resposta.status_code == 404:
            print(f"Pokémon '{nome}' não encontrado.")
        else:
            print("Erro HTTP:", resposta.status_code)
    except requests.exceptions.Timeout:
        print("A API demorou demais para responder.")
    except requests.exceptions.ConnectionError:
        print("Falha de conexão. Verifique sua Internet.")
    return None


for nome in ["Charizard", "pikachuu"]:
    dados = buscar_pokemon(nome)
    if dados:
        print(dados["name"], "->", dados["id"])
```

```
charizard -> 6
Pokémon 'pikachuu' não encontrado.
```

> **Uso consciente de APIs públicas**
>
> A PokeAPI é gratuita e mantida por voluntários. A documentação pede que os clientes guardem localmente (em *cache*) os dados já consultados, em vez de repetir as mesmas requisições. Evite laços que façam centenas de requisições sem necessidade.

Exercícios – Parte 1

- **Comparador:** peça dois nomes ao usuário e mostre qual dos dois tem o maior valor de `speed`.
- **Por tipo:** consulte `/type/electric` e liste os 10 primeiros Pokémon do tipo. Dica: os nomes estão em `dados["pokemon"][i]["pokemon"]["name"]`.
- **Paginação:** percorra as páginas seguindo o campo `next` até reunir os 60 primeiros nomes.

APIs abertas x APIs com chave

A PokeAPI não sabe quem está fazendo as requisições. Muitas APIs, porém, precisam identificar o cliente para controlar limites de uso, cobrar por planos ou bloquear abusos. Para isso, fornecem uma **chave de API** (*API key* ou *token*), enviada a cada requisição.

	PokeAPI	**REST Countries**
Cadastro	Não	Sim (plano gratuito)
Identificação do cliente	Nenhuma	Cabeçalho `Authorization: Bearer <chave>`
Sem credencial	Funciona normalmente	**401 Unauthorized**
Limites	Uso consciente (cache)	Limite mensal do plano e até 20 requisições a cada 10 s

Parte 2 – REST Countries: uma API que exige chave

A REST Countries fornece dados normalizados de mais de 250 países: nomes, códigos ISO, capitais, moedas, idiomas, fronteiras, bandeiras, população, etc. A versão atual (v5) tem a URL base:

```
https://api.restcountries.com/countries/v5
```

> **Atenção: versões antigas**

2.1 Testando sem chave

Antes de criar a conta, veja o que acontece quando não nos identificamos:

```
import requests

url = "https://api.restcountries.com/countries/v5/codes.alpha_2/BR"
resposta = requests.get(url, timeout=10)

print(resposta.status_code)
print(resposta.json())
```

```
401
{'errors': [{'message': 'Authorization key required.', 'code': 'authKeyMissing'}]}
```

O código **401 Unauthorized** indica que o servidor não sabe quem somos. Repare também no formato do erro: uma lista em `"errors"`, cada item com uma `"message"`. Conhecer esse formato nos permite exibir mensagens úteis ao usuário.

2.2 Usando a chave de demonstração

A API oferece uma chave pública de teste, `rc_live_demo`, que não exige conta. Ela sempre devolve um **exemplo fixo** (o Canadá), qualquer que seja o país pedido, mas já mostra o formato real da resposta. A chave vai no cabeçalho `Authorization`, precedida da palavra `Bearer`:

```
import requests

url = "https://api.restcountries.com/countries/v5/codes.alpha_2/BR"
cabecalhos = {"Authorization": "Bearer rc_live_demo"}

resposta = requests.get(url, headers=cabecalhos, timeout=10)
resposta.raise_for_status()
corpo = resposta.json()

print(corpo.keys())
print(corpo["data"].keys())

pais = corpo["data"]["objects"][0]
print(pais["names"]["common"], "-", pais["capitals"][0]["name"])
```

```
dict_keys(['data'])
dict_keys(['_demo', 'objects', 'meta'])
Canada - Ottawa
```

A estrutura da resposta é sempre a mesma: os países ficam na lista `corpo["data"]["objects"]` (mesmo quando há um só) e as informações de paginação ficam em `corpo["data"]["meta"]`.

2.3 Obtendo e guardando a sua chave

1. Acesse restcountries.com e clique em *Get an API key*.
2. Crie a conta no plano gratuito (*Free*) e copie a chave gerada no painel *API Keys*.
3. Guarde a chave em uma **variável de ambiente**, e nunca no código-fonte.

```
# Linux/macOS (vale para o terminal atual)
export RESTCOUNTRIES_KEY="cole_sua_chave_aqui"

# Windows (PowerShell)
$env:RESTCOUNTRIES_KEY = "cole_sua_chave_aqui"
```

> **Por que não escrever a chave no código?**
>
> Como vimos na seção 1.5 da unidade, a chave identifica **você**. Se ela for parar em um repositório público ou em uma entrega de atividade, outra pessoa poderá usar sua cota ou ter sua conta bloqueada no seu lugar. Se isso acontecer, revogue a chave no painel e gere outra.

resposta muito menor:

```
import os

import requests

BASE = "https://api.restcountries.com/countries/v5"

chave = os.environ["RESTCOUNTRIES_KEY"]
cabecalhos = {"Authorization": f"Bearer {chave}"}
parametros = {
    "response_fields": "names.common,capitals,population,currencies,languages,region,borders"
}

resposta = requests.get(f"{BASE}/codes.alpha_2/BR", headers=cabecalhos,
                        params=parametros, timeout=10)
resposta.raise_for_status()
pais = resposta.json()["data"]["objects"][0]

capital = pais["capitals"][0]["name"] if pais["capitals"] else "-"
moedas = [f"{m['name']} ({m['code']})" for m in pais["currencies"]]
idiomas = [i["name"] for i in pais["languages"]]

print("País:", pais["names"]["common"])
print("Capital:", capital)
print(f"População: {pais['population']:,}".replace(",", "."))
print("Moedas:", ", ".join(moedas))
print("Idiomas:", ", ".join(idiomas))
print("Fronteiras:", ", ".join(pais["borders"]))
```

```
País: Brazil
Capital: Brasília
População: (valor atualizado pela API)
Moedas: Brazilian real (BRL)
Idiomas: Portuguese
Fronteiras: ARG, BOL, COL, GUF, GUY, PRY, PER, SUR, URY, VEN
```

Saída ilustrativa: a população é atualizada pela API periodicamente, por isso o valor exato pode variar. Com a chave de demonstração, o mesmo código mostraria os dados do Canadá.

2.5 Reutilizando o cabeçalho com Session

Quando todas as requisições precisam da mesma chave, repetir `headers=` em cada chamada é cansativo e fácil de esquecer. Um objeto `requests.Session` guarda cabeçalhos comuns e ainda reaproveita a conexão TCP/TLS entre requisições ao mesmo servidor, o que as torna mais rápidas:

```
import requests

BASE = "https://api.restcountries.com/countries/v5"

sessao = requests.Session()
sessao.headers.update({
    "Authorization": f"Bearer {os.environ['RESTCOUNTRIES_KEY']}",
    "Accept": "application/json",
})

# países da América do Sul (filtro por propriedade), apenas nome e população
parametros = {
    "subregion": "South America",
    "response_fields": "names.common,population",
    "limit": 100,
}
resposta = sessao.get(BASE, params=parametros, timeout=10)
resposta.raise_for_status()
corpo = resposta.json()["data"]

print("Encontrados:", corpo["meta"]["total"])
mais_populosos = sorted(corpo["objects"], key=lambda p: p["population"], reverse=True)
for pais in mais_populosos[:5]:
    print(f"{pais['names']['common']:<12} {pais['population']:>13,}")
```

Outras formas de busca descritas na documentação:

Requisição	Resultado
`GET /countries/v5/name?q=bra`	Busca por parte do nome
`GET /countries/v5/capitals/Lima`	País cuja capital é Lima
`GET /countries/v5/currencies/EUR`	Países que usam o euro
`GET /countries/v5/borders/BRA`	Países que fazem fronteira com o Brasil
`GET /countries/v5?region=Europe&limit=10&offset=10`	Segunda página de 10 países da Europa

2.6 Tratando os erros da API

Além dos erros de rede, uma API com chave tem respostas de erro próprias. As mais comuns são:

Código	Significado na REST Countries
400	Parâmetro inválido (por exemplo, `q` vazio ou `limit` acima de 100 no plano gratuito)
401	Chave ausente, inexistente, revogada ou expirada
403	Conta bloqueada (limite mensal atingido) ou campo exclusivo de plano pago
404	Caminho ou propriedade inexistente
429	Requisições demais em pouco tempo (mais de 20 a cada 10 segundos)

```
import requests

BASE = "https://api.restcountries.com/countries/v5"


def criar_sessao():
    chave = os.environ.get("RESTCOUNTRIES_KEY")
    if not chave:
        raise SystemExit("Defina a variável de ambiente RESTCOUNTRIES_KEY.")
    sessao = requests.Session()
    sessao.headers["Authorization"] = f"Bearer {chave}"
    return sessao


def consultar(sessao, caminho, **parametros):
    try:
        resposta = sessao.get(f"{BASE}/{caminho}", params=parametros, timeout=10)
    except requests.exceptions.RequestException as erro:
        print(f"Falha de comunicação: {type(erro).__name__}")
        return []

    if resposta.ok:
        return resposta.json()["data"]["objects"]

    try:
        mensagem = resposta.json()["errors"][0]["message"]
    except (ValueError, KeyError, IndexError):
        mensagem = resposta.reason

    if resposta.status_code == 401:
        print("Chave inválida ou ausente:", mensagem)
    elif resposta.status_code == 429:
        print("Limite de requisições atingido. Aguarde alguns segundos.")
    else:
        print(f"Erro {resposta.status_code}: {mensagem}")
    return []


sessao = criar_sessao()
paises = consultar(sessao, "currencies/EUR", response_fields="names.common", limit=100)
print(len(paises), "países usam o euro")
```

Note que `consultar()` sempre devolve uma lista, vazia em caso de erro, o que simplifica o código de quem a usa.

Atividade prática

Desenvolva o script **`paises.py`**, um pequeno cliente de linha de comando para a REST Countries:

1. Leia a chave da variável de ambiente `RESTCOUNTRIES_KEY` e encerre com uma mensagem clara se ela não existir.
2. Em um laço, peça ao usuário um código de país com duas letras (ex.: `BR`, `PT`, `JP`); termine quando ele digitar `sair`.
3. Mostre uma ficha com nome, capital, região, população, moedas e idiomas, usando `response_fields` para pedir apenas esses campos.
4. Liste também os **nomes** dos países vizinhos (dica: `borders/{alpha_3}` ou `codes.alpha_3/{código}`).
5. Trate 401, 404/lista vazia, 429, `Timeout` e `ConnectionError` sem que o programa seja encerrado com erro.
6. Use uma única `requests.Session` para todas as requisições.

> **Desafio (opcional)**
>
> Guarde em um dicionário as respostas já recebidas e, antes de cada requisição, verifique se o país já está nele. Repetir o mesmo código não deve gerar uma nova requisição. Essa é a ideia de *cache* recomendada pela PokeAPI.

> **Entrega**
>
> Envie apenas o arquivo `paises.py`. **Confira se sua chave não está escrita no código antes de enviar.**

Conceito	Como fizemos
Requisição GET	`requests.get(url, timeout=10)`
Ler JSON aninhado	`dados["types"][0]["type"]["name"]`
Parâmetros de consulta	`params={"limit": 5, "offset": 0}`
Autenticação por chave	`headers={"Authorization": f"Bearer {chave}"}`
Chave fora do código	`os.environ["RESTCOUNTRIES_KEY"]`
Cabeçalhos comuns	`requests.Session()`
Erros	`raise_for_status()`, `status_code` e exceções de `requests`

Documentação: PokeAPI · REST Countries · requests

Última atualização: terça-feira, 27 out. 2026, 18:23

◄ Aula 20261022 - API OpenWeatherMap

Ir para a atividade